

Report 3: Fixed-Wing Position Control

Part I: Complete Original System

Part II: Every Soaring-Mode Modification and Its Impact

PX4 Autosozaring Implementation — Defence Technical Report

Radhouene — Fixed-Wing Autosozaring Project

April 2026

Contents

List of Symbols	4
I Original Fixed-Wing Position Control System	8
1 Module Identity and Scheduling	8
2 Run() — Main Execution Loop	8
3 Sensor Poll Functions	10
3.1 airspeed_poll()	10
3.2 wind_poll()	10
3.3 manual_control_setpoint_poll()	10
3.4 vehicle_attitude_poll()	10
3.5 vehicle_control_mode_poll()	11
3.6 vehicle_command_poll()	11
4 parameters_update()	11
4.1 NPFG Parameters	12
4.2 TECS Parameters	12
5 set_control_mode_current(): Mode Decision Tree	12
6 Run() Mode Dispatch Switch	13
7 NPFG: Nonlinear Path Following Guidance	14
7.1 Core Guidance Law	14
7.2 getCorrectedNpfgRollSetpoint()	15
8 adapt_airspeed_setpoint()	15
9 get_manual_airspeed_setpoint()	16

10	getManualHeightRateSetpoint()	16
11	tecs_update_pitch_throttle()	16
12	control_auto(): Setpoint Type Dispatch	17
12.1	handle_setpoint_type()	17
12.2	Inner Switch in control_auto()	17
13	control_auto_position()	17
14	control_auto_loiter()	18
15	control_auto_velocity()	18
16	control_auto_path()	18
17	control_auto_fixed_bank_alt_hold() — GPS Failsafe Phase 1	19
18	control_auto_descend() — GPS Failsafe Phase 2	19
19	control_auto_takeoff()	19
20	control_auto_landing_straight() and _circular()	19
20.1	control_auto_landing_straight() — Mission Landing	19
20.2	control_auto_landing_circular() — Hold Pattern Landing	20
21	control_manual_altitude()	20
22	control_manual_position()	20
23	control_backtransition_heading_hold() and _line_follow()	21
23.1	heading_hold	21
23.2	line_follow	21
24	Throttle Output (Original)	21
II	Soaring-Mode Modifications and Their Physical Impact	21
25	Overview of Modified Functions	21
26	Change 1: Run() — DDS Poll, Inhibit, and Staleness Watchdog	22
26.1	DDS Poll	22
26.2	Staleness Watchdog	23
26.3	AutosoaringStatus publication, 1 Hz inhibit republish, and 2 Hz phase heart-beat	23
27	Change 2: vehicle_command_poll() — CUSTOM_0 and DO_SET_MODE	24
27.1	VEHICLE_CMD_CUSTOM_0 (Soaring CLI)	24
27.2	VEHICLE_CMD_DO_SET_MODE	25
28	Change 3: parameters_update() — Eager Glide Airspeed Push	26

29 Change 4: control_auto() — Full Soaring Dispatch Block	26
29.1 Step 1: Extract Mode Variables	26
29.2 Step 2: Altitude Floor (FW_ALT_MIN)	27
29.3 Step 3: Altitude Ceiling with Hysteresis	28
29.4 Step 4: Effective Mode Flags	28
29.5 Step 5: TECS Flag and Airspeed Injection	28
29.6 Step 6: Thermal DO_REPOSITION Command	28
29.6.1 Loiter Centre Pinning	29
29.6.2 T3: Pre-Roll Airspeed Safety	29
29.6.3 Mission Index Latch	29
29.6.4 Minimum Radius and Signed Encoding	29
29.6.5 Complete DO_REPOSITION Fields	30
29.7 Step 7: Glide DO_SET_MODE Heartbeat	30
29.8 Step 8: Thermal Exit	30
29.9 Step 9: Glide Exit — DO_SET_MODE → AUTO_MISSION	31
30 Change 5: Mode 4 — Complete NPFG Bypass	32
30.1 Why Bypass	32
30.2 Implementation	32
31 Change 6: Symmetric Throttle Ramps	32
31.1 Physical Motivation	32
31.2 Pre-Glide Thrust Cache	33
31.3 Entry Ramp (glide starts)	33
31.4 Exit Ramp (glide ends)	33
31.5 Final Throttle Selection	33
31.6 Physical Impact	33
32 Complete Modification Summary	34

List of Symbols

A superscript $(\cdot)^*$ denotes a *setpoint* (commanded value). A dot $(\dot{\cdot})$ denotes the time derivative. Bold symbols ($\mathbf{v}$) denote 2-D or 3-D vectors. A hat $(\hat{\cdot})$ (in guidance context) denotes a unit vector.

Position and Velocity

Symbol	Meaning	Unit
$\mathbf{p} = (x, y)$	Aircraft ground position in local NED North-East plane	m
$\mathbf{v}_g = (v_N, v_E)$	Aircraft ground velocity (North, East)	m/s
$\mathbf{w} = (w_N, w_E)$	Wind velocity vector (North, East)	m/s
$\mathbf{v}_a = \mathbf{v}_g - \mathbf{w}$	Aircraft airspeed vector (ground velocity minus wind)	m/s
$V_a = \ \mathbf{v}_a\ $	Airspeed magnitude	m/s
$v_{g,x}$	Body-frame forward ground speed (from estimator)	m/s
$V_{g,\min}$	Minimum forward ground speed (<code>FW_GND_SPD_MIN</code>)	m/s
h	Altitude AMSL, computed as $h = -z_{\text{NED}} + h_{\text{ref}}$	m
h_{ref}	EKF reference altitude (<code>local_pos.ref_alt</code>)	m

Attitude Angles

Symbol	Meaning	Unit
ϕ	Bank (roll) angle, positive right-wing-down	rad
ϕ^*	Roll setpoint	rad
$\phi_{\max}$	Maximum roll limit (<code>FW_R_LIM</code>)	rad
θ	Pitch angle, positive nose-up	rad
θ^*	Pitch setpoint (from TECS)	rad
ψ	Yaw (heading) angle, measured clockwise from North	rad
$\mathbf{q}$	Unit quaternion representing current attitude	—
$\mathbf{q}^*$	Attitude setpoint quaternion published to inner-loop	—
R	Direction Cosine Matrix (DCM) from body to NED	—

Airspeed

Symbol	Meaning	Unit
V_{EAS}	Equivalent airspeed: corrected for air density	m/s
V_{TAS}	True airspeed: actual speed relative to air mass	m/s
$k = k_{\text{EAS2TAS}} = \sqrt{\rho_0/\rho}$	EAS-to-TAS conversion factor, ≥ 1	—
V_{EAS}^*	EAS setpoint after <code>adapt_airspeed_setpoint()</code>	m/s
V_{trim}	Trim (cruise) EAS from performance model	m/s
$V_{\text{EAS},\min}, V_{\text{EAS},\max}$	EAS flight envelope bounds from performance model	m/s
$V_{\text{TAS},\min}, V_{\text{TAS},\max}$	Corresponding TAS bounds $= k \cdot V_{\text{EAS},\min/\max}$	m/s
k_w	Wind-based airspeed scaling coefficient (<code>FW_WIND_ARSP_SC</code>)	—

NPFG Guidance

Symbol	Meaning	Unit
$\hat{\mathbf{t}}$	Unit tangent vector along the reference path	—
$\hat{\mathbf{n}} = \hat{\mathbf{t}}^\perp$	Unit normal to the path (points left of direction of travel)	—
e_y	Signed cross-track error: positive = aircraft is right of path	m
η	Track-error angle between airspeed vector and desired heading	rad
L	NPFG look-ahead distance (scales with airspeed and period)	m
T_{NPFG}	NPFG guidance period parameter (NPFG_PERIOD)	s
ζ	NPFG damping ratio (NPFG_DAMPING)	—
a_{lat}	Lateral (horizontal) acceleration command from NPFG	m/s ²
R	Loiter circle radius (positive = clockwise)	m
$f_{\text{run}} \in [0, 1]$	NPFG can-run confidence factor (1 = full confidence)	—

TECS Interface (as seen from this module)

Symbol	Meaning	Unit
h^*	Altitude setpoint passed to TECS	m
θ_{TECS}^*	Pitch setpoint returned by <code>get_tecs_pitch()</code>	rad
$\delta_{t,\text{TECS}}^*$	Throttle returned by <code>get_tecs_thrust()</code>	—
$n_z = 1/\cos \phi$	Normal load factor injected into TECS for stall floor	—

adapt_airspeed_setpoint Notation

Symbol	Meaning	Unit
$V_{\text{EAS},\text{in}}^*$	Input EAS setpoint (from waypoint or mission)	m/s
$V_{\text{EAS},\text{out}}^*$	Slew-rate limited EAS setpoint passed to TECS	m/s
$V_{\text{EAS},\text{min},\text{wind}}$	Wind-adjusted minimum EAS = $V_{\text{EAS},\text{min}} + k_w \ \mathbf{w}\ $	m/s
r_V	Airspeed setpoint slew rate (ASPD_SP_SLEW_RATE, default 1 m/s ²)	m/s ²

Manual Control

Symbol	Meaning	Unit
δ_{pitch}	Pitch stick input, $\in [-1, +1]$	—
δ_{throttle}	Throttle stick input, $\in [-1, +1]$	—
δ_{roll}	Roll stick input, $\in [-1, +1]$	—
δ_{yaw}	Yaw stick input, $\in [-1, +1]$	—
$\dot{h}_{\text{stick}}^*$	Height rate setpoint from pitch stick	m/s
V_{stick}^*	Airspeed setpoint from throttle stick	m/s
ψ_{hold}	Latched heading in heading-hold mode	rad
$\mathbf{p}_{\text{hold}}$	Latched NE position in heading-hold mode	m

GPS Failsafe

Symbol	Meaning	Unit
T_{gpsf}	Fixed-bank loiter duration before descend (NAV_GPSF_LT, default 30 s)	s
ϕ_{gpsf}	Fixed open-loop bank angle (NAV_GPSF_R, default 15°)	deg
$\dot{h}_{\text{descent}}$	Hard-coded descent rate in GPS failsafe = -0.5 m/s	m/s

Control Mode Flags (set by set_control_mode_current)

Symbol / Name	Condition
FW_POSCTRL_MODE_AUTO	Normal auto: position + position setpoint valid
FW_POSCTRL_MODE_AUTO_TAKEOFF	Takeoff setpoint type
FW_POSCTRL_MODE_AUTO_LANDING_STRAIGHT	Land type + valid previous WP
FW_POSCTRL_MODE_AUTO_LANDING_CIRCULAR	Land type, no previous WP
FW_POSCTRL_MODE_AUTO_PATH	Offboard with velocity setpoint
FW_POSCTRL_MODE_AUTO_ALTITUDE	GPS failsafe phase 1 (fixed-bank loiter)
FW_POSCTRL_MODE_AUTO_CLIMBRATE	GPS failsafe phase 2 (descend)
FW_POSCTRL_MODE_MANUAL_ALTITUDE	Manual + altitude hold
FW_POSCTRL_MODE_MANUAL_POSITION	Manual + heading hold

Soaring-Mode Symbols (Part II)

Symbol	Meaning	Unit
ϕ_{cmd}	Commanded bank angle for thermal (bank_angle_cmd)	deg
$ \phi_{\text{cmd}} $	Magnitude of commanded bank angle	deg
R_{min}	Minimum safe loiter radius at speed V and bank ϕ	m
R_{eff}	Effective loiter radius: $\max(R_{\text{companion}}, R_{\text{min}})$	m
R_c	Companion-provided loiter radius (loiter_radius_m)	m
T_{ramp}	Throttle ramp duration (FW_GLIDE_RAMP_T, default 3 s)	s
t_s	Entry ramp start timestamp (_soaring_entry_ramp_start_us)	μs
t_e	Exit ramp start timestamp (_soaring_ramp_start_us)	μs
$\delta_{t,\text{pre}}$	TECS throttle just before glide entry (_soaring_pre_glide_thrust)	—
$s_{\text{exit}}(t)$	Exit ramp scale factor = $\min((t - t_e)/T_{\text{ramp}}, 1)$	—
$h_{\text{min}}, h_{\text{max}}$	Soaring altitude floor/ceiling (FW_ALT_MIN, FW_ALT_MAX)	m
h_{hyst}	Altitude ceiling hysteresis band (FW_ALT_HYST)	m
$V_{\text{EAS,glide}}^*$	Glide EAS setpoint injected into TECS for soaring modes	m/s
a, b	Glide polar coefficients: sink rate $w_s = aV^2 + b$	—
$V_{\text{polar}}^* = \sqrt{b/a}$	Best-glide EAS from polar (Mode 2)	m/s
$\mathbf{q}^* = \text{Euler}(\phi^*, \theta^*, \psi)$	Mode 4 attitude setpoint quaternion	—

Mathematical Operators

Operator	Meaning	Notes
<code>clamp(x, a, b)</code>	Saturate: $\max(a, \min(b, x))$	Also <code>constrain()</code> in PX4
<code>lerp(a, b, t)</code>	Linear interpolation: $(1 - t)a + tb$	$t \in [0, 1]$
<code>Euler(ϕ, θ, ψ)</code>	ZYX Euler angles to unit quaternion	Roll, pitch, yaw order
<code>atan2(y, x)</code>	Four-quadrant arctangent	Returns angle in $(-\pi, \pi]$
<code>sign(x)</code>	Signum: +1 if $x > 0$, else -1	
<code>$\ \mathbf{v}\$</code>	Euclidean norm of vector $\mathbf{v}$	
<code>ε</code>	Small positive guard value against division by zero	
<code>max(a, b), min(a, b)</code>	Standard maximum / minimum	
<code>x, <code>fabsf(x)</code></code>	Absolute value	

Part I

Original Fixed-Wing Position Control System

1 Module Identity and Scheduling

File: `src/modules/fw_pos_control/FixedwingPositionControl.cpp`

Task trigger: The module is scheduled by the `local_position` uORB topic; `Run()` is called every time a new position estimate is published (nominally 50 Hz, $\Delta t = 20$ ms).

Interval guard:

$$\Delta t_{\text{eff}} = \text{clamp}\left(\frac{t_k - t_{k-1}}{10^6}, \Delta t_{\text{min}}, \Delta t_{\text{max}}\right) \quad [0.01 \text{ s}, 0.1 \text{ s}] \quad (1)$$

The clamp prevents integration blow-up during long-pauses (start-up, debugger break) and avoids division by zero.

Output topics written each cycle (if not idle):

- `vehicle_attitude_setpoint` ($\mathbf{q}^*, \delta_t$)
- `tecs_status` (debug / logging)
- `position_controller_status`
- `autosoaring_status` (FMU $\rightarrow$ companion: `soar:off` / `inhibit` / `watchdog`)
- `landing_gear` (on state change)
- `flight_phase_estimation`

2 Run() — Main Execution Loop

`Run()` is the entry point called by the scheduler. Its complete execution order each cycle:

1. **Exit check:** if `should_exit()`, unregister callback and clean up.
2. **Parameter update:** if `parameter_update` uORB updated, call `parameters_update()`.
3. **DDS soaring poll:** check `autosoaring_control` uORB; apply inhibit/override/watchdog logic; optionally republish `autosoaring_status` at 1 Hz while the post-`soar:off` DDS inhibit is active (see Section 26).
4. **Global position:** update (ϕ, λ) from `vehicle_global_position`.
5. **Altitude reference:** $h = -z_{\text{NED}} + h_{\text{ref}}$ where h_{ref} is the EKF reference altitude.
6. **EKF reset handling** (manual modes only):
 - Altitude reset: call `_tecs.handle_alt_step()` to prevent a TECS altitude-error spike.

- XY reset: clear heading-hold flag.

7. Setpoint ingestion:

- Offboard (`flag_control_offboard_enabled`): convert `trajectory_setpoint` NED position/velocity into `position_setpoint_triplet` in lat/lon/alt. If velocity + acceleration are valid, compute signed loiter radius from lateral acceleration:

$$R = -\frac{v_x^2 + v_y^2}{\|a_{\perp}\|} \cdot \text{sign}(\hat{\mathbf{v}}_H \times \hat{\mathbf{a}}_{\perp}) \quad (2)$$

- Normal: subscribe `position_setpoint_triplet`; set validity flags for pre-v/current/next. Reset altitude FOH counter `_min_current_sp_distance_xy`.

8. **Sensor polls** (in order): `airspeed_poll()`, `manual_control_setpoint_poll()`, `vehicle_attitude_poll()`, `vehicle_command_poll()`, `vehicle_control_mode_poll()`, `wind_poll()`.

9. **Air data**: update air density ρ from `vehicle_air_data`; push density-dependent TECS parameters:

$$\dot{h}_{\text{climb,max}} = f(\rho), \quad \dot{h}_{\text{sink,min}} = f(\rho) \quad (3)$$

10. **Land detection**: update `_landed`.

11. **Vehicle status**: update; clear VTOL backtransition state if not in transition.

12. **Ground speed vector**: (v_N, v_E) from local position.

13. **Mode selection**: `set_control_mode_current()`.

14. **In-air states**: `update_in_air_states()`.

15. **TECS speed weight restore**: write `FW_T_SPDWEIGHT` back to TECS (may have been overridden in landing).

16. **NPFG period restore**: write `NPFG_PERIOD` back (may have been overridden in takeoff).

17. **Default outputs**: `flaps = 0`, `spoilers = 0`, `gear = keep`, `fw_control_yaw_wheel = false`, `reset_integral = false`.

18. **Landing/takeoff state reset** (if not in respective mode).

19. **Mode dispatch switch** (17 cases — see Section 6).

20. **Post-dispatch**: throttle slew-rate gate; `status_publish()`; `landing_status_publish()`; gear publication; actuator setpoint publish; TECS debug publish.

3 Sensor Poll Functions

3.1 airspeed_poll()

1. If FW_USE_AIRSPD enabled and `airspeed_validated` has a new message:

- Check both `calibrated_airspeed_m_s` and `true_airspeed_m_s` are finite.
- Set `_airspeed_eas` (EAS) and compute EAS-to-TAS factor:

$$k = \text{clamp}\left(\frac{V_{\text{TAS, meas}}}{V_{\text{EAS, meas}}}, 0.9, 2.0\right) \quad (4)$$

- The clamp prevents out-of-range ISA atmosphere states.
- Record `_time_airspeed_last_valid`.

2. If no update for > 1 s: mark airspeed invalid.
3. On validity change: call `_tecs.enable_airspeed(valid)` to switch TECS between airspeed and no-airspeed control paths.

3.2 wind_poll()

1. If `wind_uORB` updated: store (w_N, w_E) ; set valid if both components are finite.
2. Timeout: if last update $> \text{WIND_EST_TIMEOUT}$, invalidate.
3. If wind invalid: zero (w_N, w_E) to prevent stale wind from contaminating NPFG (which uses wind for track-keeping).

3.3 manual_control_setpoint_poll()

1. Copy latest stick input from `manual_control_setpoint`.
2. Normal mapping:

$$\text{height_rate_stick} = \delta_{\text{pitch}}, \quad \text{airspeed_stick} = \delta_{\text{throttle}} \quad (5)$$

3. Alternate mapping (FW_POS_STK_CONF bit 0 set):

$$\text{height_rate_stick} = -\delta_{\text{throttle}}, \quad \text{airspeed_stick} = \delta_{\text{pitch}} \quad (6)$$

4. Timeout: if last update > 1 s, zero both to prevent stale input integrating into height or airspeed.

3.4 vehicle_attitude_poll()

1. Read quaternion $\mathbf{q}$ from `vehicle_attitude`; read body rates from `vehicle_angular_velocity`.
2. Tailsitter correction: if VTOL tailsitter, rotate DCM by 90° pitch offset before extracting Euler angles.
3. Extract (ϕ, θ, ψ) using `Eulerf(Dcmf(q))`.

4. Compute body-frame acceleration: $\mathbf{a}_b = R^\top(a_N, a_E, a_D)^\top$. Store $\mathbf{a}_{b,x}$ for ground-speed undershoot detection.
5. Compute body-frame velocity: store $\mathbf{v}_{b,x}$.
6. Update TECS load factor:

$$n_z = \frac{1}{\cos \phi}, \quad \text{_tecs.set_load_factor}(n_z) \quad (7)$$

This feeds the banked-stall speed floor (see Report 2, M9).

3.5 vehicle_control_mode_poll()

1. If updated: check if arming state just changed (was `flag_armed = false`, now `true`).
2. On arming: call `reset_takeoff_state()` and `reset_landing_state()` to flush stale integrator states.

3.6 vehicle_command_poll()

This function processes vehicle commands from uORB in a while-loop (multiple commands may arrive between cycles).

DO_GO_AROUND Aborts landing if in `FW_POSCTRL_MODE_AUTO` with `SETPOINT_TYPE_LAND` and position control is enabled. Calls `updateLandingAbortStatus(ABORTED_BY_OPERATOR)`, which sets a flag that makes `control_auto_loiter()` fly level until above a clearance altitude buffer.

DO_CHANGE_SPEED If `param1 = SPEED_TYPE_AIRSPPEED` and `param2 > 0`:

- Auto mode: overrides `pos_sp_triplet.current.cruising_speed` (used by `adapt_airspeed_set`)
- Manual mode: stores `_commanded_manual_airspeed_setpoint` (used by `get_manual_airspeed`)

DO_SET_MODE In the original system: no specific handling in `FixedwingPositionControl` (handled by Commander).

4 parameters_update()

Called on every `parameter_update` event. Pushes all parameter values into submodule instances.

4.1 NPFG Parameters

PX4 Parameter	NPFG Method
NPFG_PERIOD	setPeriod()
NPFG_DAMPING	setDamping()
NPFG_EN_PERIOD_LB/UB	enablePeriodLB/UB()
NPFG_EN_MIN_GSP	enableMinGroundSpeed()
NPFG_EN_TRACK_KEEPING	enableTrackKeeping()
NPFG_EN_WIND_REG	enableWindExcessRegulation()
FW_GND_SPD_MIN	setMinGroundSpeed()
NPFG_ROLL_TC	setRollTimeConst()
FW_R_LIM	setRollLimit() (converted to rad)
NPFG_SW_DST_MULT	setSwitchDistanceMultiplier()
NPFG_PERIOD_SF	setPeriodSafetyFactor()

4.2 TECS Parameters

PX4 Parameter	TECS Method
FW_T_CLMB_MAX (via perf model, ρ)	set_max_climb_rate()
FW_T_SINK_MAX	set_max_sink_rate()
min sink rate (via perf model, ρ)	set_min_sink_rate()
FW_T_SPDWEIGHT	set_speed_weight()
trim/min/max EAS (via perf model)	set_equivalent_airspeed_*
FW_T_THR_DAMPING	set_throttle_damp()
FW_T_THR_INTEG	set_integrator_gain_throttle()
FW_T_I_GAIN_PIT	set_integrator_gain_pitch()
FW_THR_SLEW_MAX	set_throttle_slewrates()
FW_T_VERT_ACC	set_vertical_accel_limit()
FW_T_RLL2THR	set_roll_throttle_compensation()
FW_T_PTCH_DAMP	set_pitch_damping()
FW_T_H_ERROR_TC	set_altitude_error_time_constant()
FW_T_FAST_ALT_ERR	set_fast_descend_altitude_error()
FW_T_HRATE_FF	set_altitude_rate_ff()
FW_T_TAS_ERROR_TC	set_airspeed_error_time_constant()
STE_RATE_TC	set_ste_rate_time_const()
SEB_RATE_FF	set_seb_rate_ff_gain()
ASPD_FMT_STD_DEV	set_airspeed_measurement_std_dev()
ASPD_RATE_STD_DEV	set_airspeed_rate_measurement_std_dev()
ASPD_PROC_NOISE	set_airspeed_filter_process_std_dev()

The performance model (`FixedwingPerformanceModel`) computes density-dependent limits from the aerodynamic polars and `FW_T_CLMB_MAX`, `FW_T_SINK_MIN`. On every `parameters_update` call, `_performance_model.runSanityChecks()` validates that `min < trim < max` air-speed.

5 set_control_mode_current(): Mode Decision Tree

This function runs every cycle *after* all polls, selecting one of eight control modes:

Mode	Condition
FW_POSCTRL_MODE_OTHER	Rotary-wing vehicle type without transition, or no matching condition.
FW_POSCTRL_MODE_AUTO_PATH	Offboard enabled + position enabled + finite NED velocity setpoint.
FW_POSCTRL_MODE_AUTO	Auto + position enabled + valid current setpoint (or IDLE type).
FW_POSCTRL_MODE_AUTO_TAKEOFF	As above, but setpoint type is TAKEOFF (fixed-wing only).
FW_POSCTRL_MODE_AUTO_LANDING_STRAIGHT	As above, type LAND + prev setpoint valid.
FW_POSCTRL_MODE_AUTO_LANDING_CIRCULAR	As above, type LAND + prev not valid.
FW_POSCTRL_MODE_AUTO_ALTITUDE	Auto + climb-rate enabled + armed + position <i>not</i> valid; GPS failsafe phase 1 (fixed-bank loiter for NAV_GPSF_LT seconds).
FW_POSCTRL_MODE_AUTO_CLIMBRATE	Same conditions, after NAV_GPSF_LT timeout; GPS failsafe phase 2 (emergency descent).
FW_POSCTRL_MODE_MANUAL_POSITION	Manual + position enabled.
FW_POSCTRL_MODE_MANUAL_ALTITUDE	Manual + altitude enabled.
FW_POSCTRL_MODE_TRANSITION_TO_HOVER_*	VTOL back-transition.

VTOL takeoff bypass: If VTOL and in transition while setpoint type is TAKEOFF, the type is rewritten to POSITION and mode is set to FW_POSCTRL_MODE_AUTO to use the standard waypoint follower instead of the launch sequence.

Arming guard: Failsafe altitude/climbrate modes only activate when the vehicle is armed (`flag_armed = true`) to prevent triggering during pre-arm checks.

6 Run() Mode Dispatch Switch

After `set_control_mode_current()`, a switch dispatches to the appropriate control function:

```

1  switch (_control_mode_current) {
2      case FW_POSCTRL_MODE_AUTO:
3          control_auto(...);                break;
4      case FW_POSCTRL_MODE_AUTO_ALTITUDE:
5          control_auto_fixed_bank_alt_hold(...); break;
6      case FW_POSCTRL_MODE_AUTO_CLIMBRATE:
7          control_auto_descend(...);         break;
8      case FW_POSCTRL_MODE_AUTO_LANDING_STRAIGHT:
9          control_auto_landing_straight(...); break;
10     case FW_POSCTRL_MODE_AUTO_LANDING_CIRCULAR:
11         control_auto_landing_circular(...); break;
12     case FW_POSCTRL_MODE_AUTO_PATH:
13         control_auto_path(...);             break;
14     case FW_POSCTRL_MODE_AUTO_TAKEOFF:
15         control_auto_takeoff(...);         break;
16     case FW_POSCTRL_MODE_MANUAL_POSITION:
17         control_manual_position(...);      break;
18     case FW_POSCTRL_MODE_MANUAL_ALTITUDE:

```

```

19     control_manual_altitude(...);           break;
20 case FW_POSCTRL_MODE_TRANSITION_TO_HOVER_LINE_FOLLOW:
21     control_backtransition_line_follow(...); break;
22 case FW_POSCTRL_MODE_TRANSITION_TO_HOVER_HEADING_HOLD:
23     control_backtransition_heading_hold(); break;
24 }

```

7 NPFG: Nonlinear Path Following Guidance

7.1 Core Guidance Law

NPFG computes a lateral acceleration command based on the error between the current airspeed vector and the desired track direction.

Track-error angle η : For a line segment $A \rightarrow B$ with unit tangent $\hat{\mathbf{t}}$:

$$e_y = \hat{\mathbf{n}} \cdot (\mathbf{p} - \mathbf{A}) \quad (\text{signed cross-track error}) \quad (8)$$

$$\eta = \text{atan2}(\mathbf{v}_a, \hat{\mathbf{t}}) + \arcsin\left(\text{clamp}\left(\frac{-e_y}{L}, -1, 1\right)\right) \quad (9)$$

where L is the look-ahead distance.

Look-ahead length:

$$L = \frac{T_{\text{NPFG}} V_a}{2\pi/\zeta} = f(\text{NPFG period, damping}) \quad (10)$$

Bounded below by a minimum ground speed term and above by a maximum.

Lateral acceleration command:

$$a_{\text{lat}} = \frac{2V_a^2}{L} \sin \eta \quad (11)$$

For loiter circle: The centripetal acceleration is added:

$$a_{\text{lat}} += \text{sign}(d) \cdot \frac{V_a^2}{R} \quad (12)$$

where $d = \pm 1$ for CW/CCW.

Roll setpoint:

$$\phi^* = \arctan\left(\frac{a_{\text{lat}}}{g}\right) \quad (13)$$

Clamped to $[-\phi_{\text{max}}, \phi_{\text{max}}]$ from FW_R_LIM.

7.2 getCorrectedNpfgRollSetpoint()

NPFG reports a “can-run factor” $f_{\text{run}} \in [0, 1]$ that measures confidence based on:

- Velocity estimate validity (`xy_valid` in local position).
- Wind estimate validity (`_wind_valid`).

The corrected roll:

$$\phi_{\text{corrected}}^* = f_{\text{run}} \cdot \phi_{\text{NPFG}}^* \quad (14)$$

If $f_{\text{run}} < 0.9$ for more than `ROLL_WARNING_TIMEOUT` (2s), a user warning event is emitted. This prevents the aircraft from making large rolls in GPS-degraded conditions.

Roll slew rate: After NPFG, the roll setpoint is filtered through a slew-rate limiter (`FW_PN_R_SLEW_MAX`, default 90 deg/s) to prevent step demands.

8 adapt_airspeed_setpoint()

Called before every TECS invocation. Six processing steps:

1. **Wind-based minimum airspeed scaling:** If airspeed valid, wind valid, and `FW_WIND_ARSP_SC` > 0:

$$V_{\text{EAS},\text{min}} \leftarrow \min(V_{\text{EAS},\text{min}} + k_w \|\mathbf{w}\|, V_{\text{EAS},\text{max}}) \quad (15)$$

This increases the minimum airspeed in strong winds to maintain climb performance and avoid stall.

2. **NaN/zero setpoint fallback:** If the mission waypoint cruising speed is not finite or ≤ 0 : $V_{\text{EAS}}^* \leftarrow V_{\text{trim}}$.
3. **Ground-speed undershoot correction:** If wind is *not* valid (cannot estimate it):

$$\text{if } v_{g,x} < V_{g,\text{min}} : \quad V_{\text{EAS}}^* += V_{g,\text{min}} - v_{g,x} \quad (16)$$

where $v_{g,x}$ is the body-frame forward ground speed. This prevents the aircraft being blown backwards.

4. **Bounds:** $V_{\text{EAS}}^* = \text{clamp}(V_{\text{EAS}}^*, V_{\text{EAS},\text{min}}, V_{\text{EAS},\text{max}})$.
5. **Slew-rate controller initialisation:** On first call or after TECS reset: $V_{\text{slew}}^* = \max(V_{\text{EAS},\text{min}}, V_{\text{EAS},\text{meas}})$. If the current state is below $V_{\text{EAS},\text{min}}$ (e.g. minimum changed), force the slew state up to the new minimum.
6. **Slew-rate update:** $V_{\text{out}}^* = \text{SlewRate.update}(V_{\text{EAS}}^*, \Delta t)$ with rate limit `ASPD_SP_SLEW_RATE` (default 1 m/s²).

9 get_manual_airspeed_setpoint()

Converts the throttle stick position $\delta_T \in [-1, 1]$ to an EAS setpoint for manual modes:

- **Manual airspeed enabled** (FW_POS_STK_CONF bit 1): piecewise linear interpolation:

$$V^* = \text{interpolateNXY}(\delta_T, \{-1, 0, 1\}, \{V_{\min}, V_{\text{trim}}, V_{\max}\}) \quad (17)$$

Neutral stick ($\delta_T = 0$) gives trim airspeed.

- **Airspeed override** (DO_CHANGE_SPEED received): use the commanded value clamped to $[V_{\min}, V_{\max}]$.
- **Default**: return trim airspeed.

10 getManualHeightRateSetpoint()

Converts pitch-stick position to a height rate setpoint for manual altitude control:

$$\dot{h}^* = \begin{cases} \delta_{\text{pitch}} \cdot \dot{h}_{\text{climb,target}} & \delta_{\text{pitch}} > 0 \\ -\delta_{\text{pitch}} \cdot \dot{h}_{\text{sink,target}} & \delta_{\text{pitch}} < 0 \\ \text{NaN} & |\delta_{\text{pitch}}| < \varepsilon_{\text{dead}} \end{cases} \quad (18)$$

NaN output means “no height-rate control” (TECS tracks altitude reference).

11 tecs_update_pitch_throttle()

Single call site for TECS. Called by every position control sub-mode.

Step 1: Inject limits into TECS.

```

1 _tecs.set_min_pitch(pitch_min_rad);
2 _tecs.set_max_pitch(pitch_max_rad);
3 _tecs.set_throttle_min(thr_min);
4 _tecs.set_throttle_max(thr_max);
5 _tecs.set_target_climbrate(target_climbrate);
6 _tecs.set_target_sinkrate(target_sinkrate);

```

Step 2: Low-height mode. If `is_low_height` (aircraft below `FW_LH_APC_FACTOR` $\times$ landing approach clearance altitude), TECS uses tighter altitude error gains and smaller pitch limits to improve precision near the ground.

Step 3: Underspeed handling. If `disable_underspeed_handling = false` (default), the `detect_underspeed_enabled` flag is true in TECS.

Step 4: Height-rate override. If the caller provides a finite `height_rate_setpoint`, it is passed to the TECS reference model as a direct height-rate command (bypassing the altitude proportional controller).

Step 5: EAS $\rightarrow$ TAS setpoint. TECS receives the raw EAS setpoint; internally calls `calcTrueAirspeedSetpoint()` (see Report 2).

Step 6: Call `TECS::update()`.

Step 7: Output.

- `get_tecs_pitch()` returns the TECS pitch setpoint θ^* .
- `get_tecs_thrust()` returns the TECS throttle δ_t^* clamped to $[\delta_{t,\min}, \delta_{t,\max}]$.

12 control_auto(): Setpoint Type Dispatch

control_auto() runs when in FW_POSCTRL_MODE_AUTO.

12.1 handle_setpoint_type()

Before the inner switch, the setpoint type is re-evaluated:

- If velocity control only (no position): return SETPOINT_TYPE_VELOCITY.
- If type is POSITION and not approaching VTOL backtransition: compute 3D distance to waypoint. If aircraft is within the acceptance radius AND has the correct vertical separation:

$$d_{xy} < r_{acc}, \quad d_z > r_{z,acc} \Rightarrow \text{switch to LOITER} \quad (19)$$

r_{acc} comes from npfg.switchDistance(500) which scales with airspeed.

12.2 Inner Switch in control_auto()

```
1 switch (position_sp_type) {
2   case SETPOINT_TYPE_IDLE:      thrust = 0; break;
3   case SETPOINT_TYPE_POSITION:
4     control_auto_position(...); break;
5   case SETPOINT_TYPE_VELOCITY:
6     control_auto_velocity(...); break;
7   case SETPOINT_TYPE_LOITER:
8     control_auto_loiter(...);   break;
9   #ifdef CONFIG_FIGURE_OF_EIGHT
10    case SETPOINT_TYPE_FIGURE_EIGHT:
11      controlAutoFigureEight(...); break;
12  #endif
13 }
```

13 control_auto_position()

1. **Altitude First-Order Hold (FOH):** Linearly interpolates the altitude setpoint between previous and current waypoints as the aircraft approaches:

$$h^* = h_{prev} + \frac{h_{curr} - h_{prev}}{d_{A \rightarrow B} - r_{acc}} \cdot (d_{A \rightarrow B} - d_{min}) \quad (20)$$

where d_{min} is the smallest distance to the current waypoint ever achieved. This ensures the aircraft reaches the waypoint altitude precisely at the acceptance radius rather than demanding a step change.

2. **NPFG guidance:** Calls navigateWaypoints(prev, curr, ...) if a valid previous waypoint exists; else navigateWaypoint(curr, ...). Outputs roll setpoint ϕ^* .
3. **Airspeed setpoint:** adapt_airspeed_setpoint(pos_sp_curr.cruising_speed, ...). After NPFG: $V^* \leftarrow \text{getAirspeedRef}() / k$ (NPFG may increase the airspeed demand to maintain track in crosswinds).

4. **TECS call:** `tecs_update_pitch_throttle( $h^*$ ,  $V^*$ , ...)`.
5. **Attitude setpoint:** $\mathbf{q}^* = \text{Euler}(\phi_{\text{NPFG}}^*, \theta_{\text{TECS}}^*, \psi)$.

14 `control__auto__loiter()`

1. **Radius:** from `pos_sp_curr.loiter_radius`; if zero, use `NAV_LOITER_RAD` (default 50 m).
2. **Early landing configuration:** If the next setpoint is type `LAND` and the aircraft is within $R + r_{\text{switch}}$ of the loiter circle centre, and `FW_LND_EARLYCFG` enabled:
 - Set airspeed to landing airspeed `FW_LND_AIRSPD` (or stall margin if not set).
 - Deploy flaps: `_flaps_setpoint = FW_FLAPS_LND_SCL`.
 - Deploy spoilers: `_spoilers_setpoint = FW_SPOILERS_LND`.
 - Lower gear: `GEAR_DOWN`.
 - Enable low-height TECS mode.
3. **Landing abort override:** If `_landing_abort_status` is set:
 - If altitude delta to loiter $< \text{kClearanceAltitudeBuffer}$ (10 m): clear abort status (done).
 - Else: set roll to 0 (fly straight), retract flaps if airspeed is safe.
4. **NPFG:** `navigateLoiter(centre, pos, R, direction, ...)` uses the loiter-circle guidance.
5. **TECS call:** with loiter altitude setpoint.
6. **Attitude setpoint:** $\mathbf{q}^* = \text{Euler}(\phi_{\text{NPFG}}^*, \theta_{\text{TECS}}^*, \psi)$.

15 `control__auto__velocity()`

Used for offboard velocity setpoints (no position, only velocity).

1. Target velocity from `pos_sp_curr.vx/vy`; heading from `atan2(vy, vx)`.
2. NPFG navigates toward a virtual waypoint in the target direction.
3. TECS controls altitude to `pos_sp_curr.alt`.

16 `control__auto__path()`

Used for offboard path following with position + velocity + acceleration.

1. If acceleration is available: compute lateral curvature from the cross-product of normalised velocity and acceleration; use signed loiter radius from centripetal acceleration.
2. NPFG path follows the straight segment defined by velocity direction.
3. TECS controls altitude.

17 control_auto_fixed_bank_alt_hold() — GPS Failsafe Phase 1

Activated when the GPS position estimate is lost (FW_POSCTRL_MODE_AUTO_ALTITUDE) for up to NAV_GPSF_LT seconds (default 30s).

- Roll set to NAV_GPSF_R (fixed open-loop bank angle, default 15°): aircraft circles indefinitely.
- Yaw: 0 (heading holds whatever it was).
- TECS controls altitude to current measured altitude, trim airspeed.
- Special: if NED z or v_z is invalid (no barometer): thrust forced to FW_THR_MIN to avoid climb-away.

18 control_auto_descend() — GPS Failsafe Phase 2

Activated after NAV_GPSF_LT timeout.

- Height rate setpoint hard-coded to -0.5 m/s .
- Same fixed-bank open-loop roll as Phase 1.
- TECS passed this height rate directly (bypasses altitude controller).
- If v_z invalid: thrust forced to minimum.

19 control_auto_takeoff()

Multi-phase launch sequence (catapult, runway, VTOL front-transition):

1. **Pre-launch:** no airspeed, no attitude control; wait for acceleration signature or RC trigger.
2. **Climbout:** TECS active, pitch clamped above FW_TKO_PITCH_MIN to ensure ground clearance. NPFG follows the runway heading (or waypoint).
3. **Transition altitude reached:** gear up; switch to normal auto-position mode.

20 control_auto_landing_straight() and _circular()

20.1 control_auto_landing_straight() — Mission Landing

1. Uses previous WP as the glideslope start; current WP as touchdown.
2. Glideslope: altitude setpoint linearly interpolated from approach altitude to runway elevation.
3. Flare: when within FW_LND_FLALT of touchdown:

- Throttle cut (FW_THR_MIN).
- Pitch held at FW_LND_FL_PMAX.
- Airspeed setpoint switches to FW_LND_AIRSPD.

20.2 control_auto_landing_circular() — Hold Pattern Landing

Used when no previous waypoint is available (e.g. Return-to-Land mode). The aircraft spirals down in a loiter until below the decision altitude, then transitions to straight final.

21 control_manual_altitude()

1. `updateManualTakeoffStatus()`: detect first climb above takeoff threshold; set `_completed_manual_takeoff`.
2. Airspeed setpoint: `adapt_airspeed_setpoint(get_manual_airspeed_setpoint(), ...)`.
3. Height rate: `getManualHeightRateSetpoint()` from pitch stick.
4. Pitch minimum: FW_P_LIM_MIN normally; or MIN_PITCH_DURING_MANUAL_TAKEOFF (8°) if not yet completed takeoff.
5. Throttle maximum: if on ground and stick below THROTTLE_THRESH (0.05): $\delta_{t,\max} = 0$ (kill switch).
6. Roll: direct from stick scaled by FW_R_LIM.
7. TECS + attitude setpoint.

22 control_manual_position()

Manual position mode adds heading hold on top of manual altitude:

1. Same airspeed and height-rate processing as altitude mode.
2. **Heading hold engagement condition:** Stick roll and yaw both below HDG_HOLD_MAN_INPUT_TH (0.01) AND yaw rate below HDG_HOLD_YAWRATE_THRESH (0.15 rad/s).
3. When engaged:
 - Record hold yaw ψ_{hold} and hold position $\mathbf{p}_{\text{hold}}$.
 - NPFG `navigateLine()` from hold position along heading ψ_{hold} .
 - Roll from NPFG; yaw from ψ_{hold} .
4. EKF XY reset during heading hold: update hold position for 2 s to let EKF converge.
5. When stick moves: disable heading hold, use direct roll stick.

23 control_backtransition_heading_hold() and _line_follow()

Used for VTOL back-transition (forward flight → hover).

23.1 heading_hold

- Lock the heading at transition start.
- Simulate wind = 0; ground speed = airspeed aligned with heading.
- NPFG navigates a virtual waypoint 200 m ahead.
- Thrust forced to FW_THR_MIN (overridden by transition logic in VTOL module).

23.2 line_follow

- NPFG follows the line from back-transition start position to land waypoint.
- Position at back-transition start captured on first call.

24 Throttle Output (Original)

After the mode dispatch, throttle is applied:

```
1 if (position_sp_type == SETPOINT_TYPE_IDLE) {  
2     _att_sp.thrust_body[0] = 0.0f;  
3 } else if (_landed && ...) {  
4     _att_sp.thrust_body[0] = min(_param_fw_thr_idle.get(), 1.f);  
5 } else {  
6     _att_sp.thrust_body[0] = min(get_tecs_thrust(), _param_fw_thr_max.  
7         get());  
}
```

No ramp logic. TECS output goes directly to the airframe actuators.

Part II

Soaring-Mode Modifications and Their Physical Impact

25 Overview of Modified Functions

The soaring implementation modifies the following functions:

Function	What changes
Run()	DDS poll, inhibit/watchdog, status republish, DDS→cmd arbiter
vehicle_command_poll()	CUSTOM_0, DO_SET_MODE, publish_autosoaring_status
publish_autosoaring_status()	uORB autosoaring_status (bridged to ROS 2)
parameters_update()	Glide airspeed eager pre-push
control_auto()	Full soaring dispatch block
Throttle output	Symmetric entry/exit ramps

26 Change 1: Run() — DDS Poll, Inhibit, and Staleness Watchdog

26.1 DDS Poll

```

1  autosoaring_control_s soaring_msg;
2  if (_autosoaring_control_sub.update(&soaring_msg)) {
3      const hrt_abstime now = hrt_absolute_time();
4      const bool dds_inhibited = (_soaring_dds_inhibit_until_us > 0
5          && now < _soaring_dds_inhibit_until_us);
6      const bool dds_wants_soaring =
7          (soaring_msg.soaring_mode != autosoaring_control_s::SOARING_OFF)
8          ;
9
10     if (dds_inhibited && dds_wants_soaring) {
11         // ignore: CLI soar:off inhibit active
12     } else {
13         if (!dds_wants_soaring) {
14             _soaring_dds_inhibit_until_us = 0; // DDS OFF cancels
15             inhibit
16         }
17         if (!_soaring_local_override) {
18             const bool prev_thermal = (
19                 _autosoaring_control.soaring_mode ==
20                 autosoaring_control_s::SOARING_THERMAL_LOITER ||
21                 _autosoaring_control.soaring_mode ==
22                 autosoaring_control_s::SOARING_THERMAL_BANK);
23             const bool new_thermal = (
24                 soaring_msg.soaring_mode == autosoaring_control_s::
25                 SOARING_THERMAL_LOITER ||
26                 soaring_msg.soaring_mode == autosoaring_control_s::
27                 SOARING_THERMAL_BANK);
28             _autosoaring_control = soaring_msg;
29             _autosoaring_last_recv_us = now;
30             if (!prev_thermal && new_thermal) {
31                 _soaring_mode_cmd_last_us = 0;
32                 _soaring_last_lat = NAN;
33                 _soaring_last_lon = NAN;
34             }
35         }
36     }
37 }

```

DDS inhibit: When `soar:off` is issued from CLI, the DDS path is blocked for 5s (`_soaring_dds_inhibit_until_us = now + 5s`). This prevents an autonomous companion from immediately re-enabling soaring after the operator has manually stopped it. If the companion publishes `soaring_mode = OFF` while the inhibit is active, the remaining cooldown is cleared immediately (`_soaring_dds_inhibit_until_us = 0`).

Authority arbitration: The `_soaring_local_override` flag gives CLI exclusive authority. When set, DDS messages are received but do *not* overwrite `_autosoaring_control`. CLI and DDS cooperate: CLI sets the mode; DDS can update ancillary fields (loiter position, bank angle) while CLI holds authority.

26.2 Staleness Watchdog

```

1  if (!_soaring_local_override &&
2      _autosoaring_last_recv_us > 0 &&
3      (hrt_absolute_time() - _autosoaring_last_recv_us) > 2_s) {
4      if (_autosoaring_control.soaring_mode != autosoaring_control_s::
5          SOARING_OFF) {
6          _autosoaring_control.soaring_mode = autosoaring_control_s::
7              SOARING_OFF;
8          publish_autosoaring_status(autosoaring_status_s::
9              SOURCE_STALENESS_WATCHDOG,
10              autosoaring_control_s::SOARING_OFF, 0);
11      }
12      _tecs.set_gliding_mode_enabled(false);
13      _autosoaring_last_recv_us = 0;
14  }

```

Physical motivation: The companion computer may crash or lose its DDS connection while in mid-glide. Without a watchdog, the aircraft would remain engine-off indefinitely at risk of altitude loss. The 2s timeout restores powered flight automatically.

Not triggered by CLI: `_autosoaring_last_recv_us = 0` when CLI is the authority (set in `vehicle_command_poll()` for `CUSTOM_0` commands). The watchdog only fires if `_autosoaring_last_recv_us > 0`, which is only set when DDS delivered a message.

26.3 AutosoaringStatus publication, 1 Hz inhibit republish, and 2 Hz phase heartbeat

On CLI `soar:off` (`CUSTOM_0` with mode 0), after TECS gliding is disabled, `publish_autosoaring_status` is called with `SOURCE_CLI_SOARING_OFF`, the effective `soaring_mode` (`OFF`), `dds_inhibit_until_us` equal to `hrt_absolute_time() + 5s`, and `fmu_soaring_phase = SOARING_PHASE_CRUISE`. The timestamp `_autosoaring_status_repub_us` is initialised so that, for the duration of the inhibit window, the same status is republished at 1 Hz (companion may miss a single DDS sample):

```

1  if (_soaring_dds_inhibit_until_us > 0 && status_now <
2      _soaring_dds_inhibit_until_us) {
3      if (status_now - _autosoaring_status_repub_us >= 1_s) {
4          _autosoaring_status_repub_us = status_now;
5          publish_autosoaring_status(SOURCE_CLI_SOARING_OFF,
6              _autosoaring_control.soaring_mode,
7              _soaring_dds_inhibit_until_us,

```

```

7         SOARING_PHASE_CRUISE);
8     }
9 } else {
10     _autosoaring_status_repub_us = 0;
11 }

```

In addition, a 2 Hz periodic heartbeat is published at the end of every `control_auto()` call (inside the position-control rate loop at ≈ 50 Hz), derived from the current `effective_glide/` `effective_thermal` booleans:

```

1 if (phase_now - _autosoaring_status_periodic_us >= 500_ms) {
2     _autosoaring_status_periodic_us = phase_now;
3     uint8_t fmu_phase;
4     if (effective_thermal)
5         fmu_phase = PX4_ISFINITE(_soaring_last_lat)
6             ? SOARING_PHASE_THERMAL : SOARING_PHASE_SEARCHING;
7     else if (effective_glide)
8         fmu_phase = SOARING_PHASE_GLIDING;
9     else
10         fmu_phase = SOARING_PHASE_CRUISE;
11     publish_autosoaring_status(SOURCE_PERIODIC,
12         _autosoaring_control.soaring_mode,
13         _soaring_dds_inhibit_until_us, fmu_phase);
14 }

```

This means `AutosoaringStatus` is **not** limited to the 5s inhibit window. It is published continuously: every 500 ms during any active soaring, at 1 Hz during the DDS inhibit cooldown, and as an immediate one-shot on event (watchdog or `soar:off`). Because uORB automatically logs every published topic, the phase sequence is available in the ULog for post-flight analysis.

The uORB topic is listed under publications in `dds_topics.yaml` as `/fmu/out/autosoaring_stat`

27 Change 2: `vehicle_command_poll()` — `CUSTOM_0` and `DO_SET_MODE`

27.1 `VEHICLE_CMD_CUSTOM_0` (Soaring CLI)

```

1 const uint8_t new_mode = constrain((int)cmd.param1,
2     0, (int)SOARING_THERMAL_BANK);
3 _autosoaring_control.soaring_mode = new_mode;
4 _autosoaring_control.loiter_radius_m =
5     (cmd.param3 > FLT_EPSILON) ? cmd.param3 : NAN;
6 if (PX4_ISFINITE(cmd.param2) && cmd.param2 > FLT_EPSILON)
7     _autosoaring_control.bank_angle_cmd = cmd.param2;

```

On enabling (`new_mode != SOARING_OFF`):

- `_soaring_local_override = true`
- Clear inhibit (`_soaring_dds_inhibit_until_us = 0`)
- Clear latch (`_soaring_forbidden_latched = false`)
- Disable watchdog (`_autosoaring_last_recv_us = 0`)

- Set “paired mode-switch” flag (`_soaring_expect_set_mode = true`)
- Reset DO_REPOSITION debounce (`_soaring_mode_cmd_last_us = 0, _soaring_last_lat = NAN`)

On disabling (`new_mode == SOARING_OFF`):

- Clear override and latch
- Block DDS for 5s (`_soaring_dds_inhibit_until_us = now + 5s`)
- Immediately call `_tecs.set_gliding_mode_enabled(false)`
- Publish `autosoaring_status` (`SOURCE_CLI_SOARING_OFF`, inhibit deadline) and seed the 1 Hz republish timer

27.2 VEHICLE_CMD_DO_SET_MODE

When CLI has override, the module must distinguish between:

1. A DO_SET_MODE *paired* with CUSTOM_0 (i.e. `soar:glide` sends both in sequence): the mode switch IS part of soaring activation. Consume `_soaring_expect_set_mode` and keep soaring on.
2. A standalone DO_SET_MODE (operator typed `commander mode auto:mission`): interpret as “exit soaring, restore powered flight”.

```

1  if (_soaring_local_override) {
2      if (_soaring_expect_set_mode) {
3          _soaring_expect_set_mode = false; // consume pairing
4      } else {
5          // Check target mode: ignore QGC heartbeats (AUTO_MISSION/LOITER)
6
7          const uint8_t main = (uint8_t)cmd.param2;
8          const uint8_t sub  = (uint8_t)cmd.param3;
9          const bool compat =
10              (main == PX4_CUSTOM_MAIN_MODE_AUTO &&
11               (sub == PX4_CUSTOM_SUB_MODE_AUTO_MISSION ||
12                sub == PX4_CUSTOM_SUB_MODE_AUTO_LOITER));
13          if (!compat) {
14              // Incompatible mode (MANUAL, STABILIZED, etc.) -> exit
15              // soaring
16              _autosoaring_control.soaring_mode = SOARING_OFF;
17              _soaring_local_override = false;
18              _tecs.set_gliding_mode_enabled(false);
19          }
20          // else: AUTO_MISSION / AUTO_LOITER heartbeat -> keep soaring
21      }
22  }

```

QGC heartbeat fix. QGC periodically broadcasts `DO_SET_MODE(AUTO_MISSION)` to maintain the displayed mode. Without the `compat` guard, every such heartbeat would arrive as a “standalone” mode-switch and immediately disable soaring — a race condition that was observed in SITL testing.

The guard keeps soaring active when the target mode is `AUTO_MISSION` or `AUTO_LOITER` (both are soaring-compatible). Any other mode (`MANUAL`, `ACRO`, `STABILIZED`, `ALTITUDE`, `POSCTL`) is considered an intentional operator override and correctly exits soaring.

Interaction with altitude floor. The QGC fix keeps `_soaring_local_override = true` permanently during CLI glide, which had the unintended side effect of also keeping the `FW_ALT_MIN` check bypassed (the old code guarded it with `!_soaring_local_override`). This was corrected by removing that guard from the altitude floor check (see Step 2 above).

28 Change 3: `parameters_update()` — Eager Glide Airspeed Push

```

1  const float polar_a = _param_fw_polar_a.get();
2  const float polar_b = _param_fw_polar_b.get();
3  const float v_polar = (polar_a > FLT_EPSILON && polar_b > FLT_EPSILON)
4    ? sqrtf(polar_b / polar_a) : 0.0f;
5  const bool polar_mode =
6    (_autosoaring_control.soaring_mode == SOARING_GLIDE_POLAR);
7  _tecs.set_gliding_airspeed_setpoint(
8    polar_mode && v_polar > FLT_EPSILON
9    ? v_polar : _param_fw_glide_airspd.get());
10 _tecs.set_glide_i_decay(_param_fw_glide_i_decay.get());
11 _tecs.set_glide_stall_eas(_param_fw_airspd_stall.get());

```

Why eager: When Mode 2 (polar) activates mid-mission, there is one TECS cycle between the DDS message arriving and `control_auto()` running. The eager push here ensures that first cycle uses the correct polar setpoint rather than the old fixed value.

FW_AIRSPD_STALL push: `set_glide_stall_eas()` stores the stall speed so that `calcTrueAirspeedSetpoint()` can use it as the glide airspeed floor instead of a hard-coded 3 m/s magic constant. `FW_AIRSPD_STALL` already exists in PX4 and is used by `fw_att_control` and `airspeed_selector`. `fw_pos_control` now reads it and forwards it to TECS on every parameter update.

29 Change 4: `control_auto()` — Full Soaring Dispatch Block

This block executes at the top of `control_auto()` before the inner setpoint-type switch.

29.1 Step 1: Extract Mode Variables

```

1  const uint8_t soaring_cmd = _autosoaring_control.soaring_mode;
2  const bool glide_cmd      = (soaring_cmd == SOARING_GLIDE_FIXED ||
3                               soaring_cmd == SOARING_GLIDE_POLAR);

```

```

4  const bool thermal_cmd      = (soaring_cmd == SOARING_THERMAL_LOITER ||
5                                soaring_cmd == SOARING_THERMAL_BANK);
6  const bool soaring_allowed = soaring_cmd != SOARING_OFF &&
7                                (_soaring_local_override || !_soaring_forbidden_latched);

```

29.2 Step 2: Altitude Floor (FW_ALT_MIN)

```

1  // Enforced on ALL paths (DDS and CLI).
2  if (soaring_requested && current_altitude < alt_min) {
3      if (!_soaring_forbidden_latched) {
4          _soaring_forbidden_latched = true;
5          if (_soaring_local_override) {
6              // CLI was in control: release override so mission resumes
7              _soaring_local_override = false;
8              _soaring_expect_set_mode = false;
9              PX4_WARN("Soaring_CLI_override_released: alt %.0f <
FW_ALT_MIN %.0f",
10                      current_altitude, alt_min);
11          } else {
12              PX4_WARN("Soaring_disabled: alt %.0f < min %.0f",
13                      current_altitude, alt_min);
14          }
15      }
16  }
17  if (!soaring_requested)
18      _soaring_forbidden_latched = false;

```

Design rationale. The latch prevents rapid cycling near the floor altitude: once `_soaring_forbidden` is set it stays set until soaring is explicitly disabled (`soaring_requested = false`).

CLI mode fix (hardware testing). The original code had:

```

1  if (!_soaring_local_override && soaring_requested && ...)

```

This skipped the check when CLI was active (`_soaring_local_override = true`). The intent was “the pilot is in control, trust them”. However, a side-effect of the QGC-heartbeat fix (which keeps `_soaring_local_override` permanently `true` during CLI glide) was that the altitude floor was *never* triggered on hardware. The aircraft descended to ground without automatic recovery.

Current design: the check fires for *all* paths. When it fires during CLI override, `_soaring_local_override` is also cleared so that:

1. TECS restores throttle immediately (next cycle: `effective_glide = false`).
2. The glide-exit block (Step 9) fires and publishes `DO_SET_MODE → AUTO_MISSION`.
3. The operator must re-issue `soar:glide` after regaining altitude.

Altitude measurement.

```

1  const float current_altitude =
2      _local_pos.z_global
3      ? -_local_pos.z + _local_pos.ref_alt
4      : _current_altitude;

```

When global-frame position is available (`z_global = true`), the GPS altitude is used. Otherwise the barometric estimate `_current_altitude` is used. Both are above mean sea level (AMSL) — set `FW_ALT_MIN` accordingly (e.g. home AMSL + minimum AGL).

29.3 Step 3: Altitude Ceiling with Hysteresis

$$h \geq h_{\max} \Rightarrow \text{_alt_max_reached} = \text{true} \quad (21)$$

$$h < h_{\max} - h_{\text{hyst}} \Rightarrow \text{_alt_max_reached} = \text{false} \quad (22)$$

Where $h_{\max} = \text{FW_ALT_MAX}$, $h_{\text{hyst}} = \text{FW_ALT_HYST}$.

When ceiling is reached during thermalling, mode degrades to glide:

$$\text{effective_thermal} = \text{false}, \quad \text{effective_glide} = \text{true} \quad (23)$$

29.4 Step 4: Effective Mode Flags

```

1  const bool effective_glide =
2      soaring_allowed && (glide_cmd || _alt_max_reached);
3  const bool effective_thermal =
4      soaring_allowed && thermal_cmd && !_alt_max_reached;
5  const bool effective_thermal_bank =
6      effective_thermal && (soaring_cmd == SOARING_THERMAL_BANK);
7  const bool currently_soaring = effective_glide || effective_thermal;

```

29.5 Step 5: TECS Flag and Airspeed Injection

```

1  _tecs.set_gliding_mode_enabled(currently_soaring);
2
3  if (soaring_cmd == SOARING_GLIDE_FIXED
4      && _autosoaring_control.airspeed_cmd > FLT_EPSILON)
5      _tecs.set_gliding_airspeed_setpoint(
6          _autosoaring_control.airspeed_cmd);           // Mode 1
7  else if (soaring_cmd == SOARING_GLIDE_POLAR) {
8      const float v = sqrtf(_param_fw_polar_b.get() /
9                          _param_fw_polar_a.get());
10     _tecs.set_gliding_airspeed_setpoint(v);           // Mode 2
11 } else
12     _tecs.set_gliding_airspeed_setpoint(
13         _param_fw_glide_airspd.get());               // Modes 3,4
14
15 // Bank angle for load-factor stall floor
16 _soaring_bank_angle_cmd_deg =
17     (PX4_ISFINITE(_autosoaring_control.bank_angle_cmd) &&
18      _autosoaring_control.bank_angle_cmd > FLT_EPSILON)
19     ? _autosoaring_control.bank_angle_cmd
20     : _param_fw_thermal_bank.get();
21 _tecs.set_load_factor(1.f / cosf(
22     radians(_soaring_bank_angle_cmd_deg)));

```

29.6 Step 6: Thermal DO_REPOSITION Command

When `effective_thermal` is true, the navigator must switch to `AUTO_LOITER`.

29.6.1 Loiter Centre Pinning

```

1  const bool dds_has_pos =
2      PX4_ISFINITE(_autosoaring_control.loiter_lat) &&
3      fabsf(_autosoaring_control.loiter_lat) > FLT_EPSILON;
4  const double lat = dds_has_pos
5      ? (double)_autosoaring_control.loiter_lat : curr_pos(0);
6  const double lon = dds_has_pos
7      ? (double)_autosoaring_control.loiter_lon : curr_pos(1);
8  const bool first_activation = !PX4_ISFINITE(_soaring_last_lat);
9  const bool pos_changed = first_activation || explicit_pos_changed;
10 const bool dir_changed = !first_activation &&
11     (_autosoaring_control.loiter_clockwise !=
12     _soaring_last_clockwise);

```

Bug fix: Without pinning, DO_REPOSITION is sent every cycle with the drifting `curr_pos`, causing a moving loiter centre. The `first_activation` flag sends exactly *one* DO_REPOSITION. Subsequent cycles: `pos_changed = false`, command not re-sent.

29.6.2 T3: Pre-Roll Airspeed Safety

On the first activation only:

$$n_z = \frac{1}{\cos |\phi_{\text{cmd}}|}, \quad V_{\text{floor}} = 1.05 \times V_{\text{stall,level}} \times \sqrt{n_z} \quad (24)$$

If $V_{\text{EAS,meas}} < V_{\text{floor}}$: delay the DO_REPOSITION. The aircraft stays in glide until speed is sufficient.

29.6.3 Mission Index Latch

Before the first DO_REPOSITION:

```

1  mission_s snap{};
2  if (_mission_sub.copy(&snap) && snap.count > 0) {
3      _soaring_mission_resume_seq = (uint16_t)constrain(
4          snap.current_seq, 0, snap.count - 1);
5      _soaring_mission_resume_valid = true;
6  }

```

29.6.4 Minimum Radius and Signed Encoding

$$R_{\text{min}} = \frac{(V_{\text{EAS,glide}} \cdot k)^2}{g \tan |\phi_{\text{cmd}}|} \quad (25)$$

$$R_{\text{eff}} = \max(\text{loiter_radius_m}, R_{\text{min}}) \quad (26)$$

$$\text{cmd.param3} = \begin{cases} +R_{\text{eff}} & \text{loiter_clockwise} = \text{true} \\ -R_{\text{eff}} & \text{loiter_clockwise} = \text{false} \end{cases} \quad (27)$$

29.6.5 Complete DO_REPOSITION Fields

Field	Value
command	VEHICLE_CMD_DO_REPOSITION
param1	-1 (keep current speed)
param2	1 (switch to loiter)
param3	$\pm R_{\text{eff}}$ (signed radius)
param4	NaN (keep yaw)
param5	loiter lat [deg]
param6	loiter lon [deg]
param7	current altitude [m AMSL]

29.7 Step 7: Glide DO_SET_MODE Heartbeat

```

1  if (effective_glide &&
2      (now - _soaring_mode_cmd_last_us) > 1_s) {
3      cmd.param2 = (float)PX4_CUSTOM_MAIN_MODE_AUTO;
4      cmd.param3 = (float)PX4_CUSTOM_SUB_MODE_AUTO_MISSION;
5      _pub_vehicle_command.publish(cmd);
6      _soaring_mode_cmd_last_us = now;
7  }
```

Sent every second to keep navigator in AUTO_MISSION while gliding.

29.8 Step 8: Thermal Exit

```

1  if (_soaring_was_thermal && !effective_thermal && !effective_glide) {
2      // switch back to mission
3      exit_cmd.param3 = (float)PX4_CUSTOM_SUB_MODE_AUTO_MISSION;
4      _pub_vehicle_command.publish(exit_cmd);
5      // clear all state
6      _soaring_last_lat = NAN;
7      _soaring_last_lon = NAN;
8      _soaring_last_clockwise = true;
9      _soaring_local_override = false;
10     // schedule mission resume next cycle
11     if (_soaring_mission_resume_valid)
12         _soaring_pending_mission_restore = true;
13 }
```

Mission restore is applied one cycle later, after the navigator has confirmed it is in AUTO_MISSION state:

```

1  if (_soaring_pending_mission_restore
2      && nav_state == NAVIGATION_STATE_AUTO_MISSION) {
3      restore_cmd.command = VEHICLE_CMD_MISSION_START;
4      restore_cmd.param1 = (float)_soaring_mission_resume_seq;
5      _pub_vehicle_command.publish(restore_cmd);
6      _soaring_pending_mission_restore = false;
7  }
```

29.9 Step 9: Glide Exit — DO_SET_MODE → AUTO_MISSION

This block is new and mirrors Step 8 for the *glide* case. When the aircraft was gliding (`_soaring_was_glide = true`) and glide just ended (altitude floor, explicit `soar:off`, or any other path), the FMU publishes a mode-switch command so the navigator re-enters mission flight:

```
1  const bool soaring_was_glide_only =
2      _soaring_was_glide && !_soaring_was_thermal;
3
4  if (soaring_was_glide_only && !effective_glide && !effective_thermal) {
5      vehicle_command_s cmd{};
6      cmd.command = VEHICLE_CMD_DO_SET_MODE;
7      cmd.param1 = 1.f; // CUSTOM_MODE_ENABLED
8      cmd.param2 = (float)PX4_CUSTOM_MAIN_MODE_AUTO;
9      cmd.param3 = (float)PX4_CUSTOM_SUB_MODE_AUTO_MISSION;
10     _pub_vehicle_command.publish(cmd);
11
12     // Clear override so vehicle_command_poll() does not intercept
13     // this DO_SET_MODE as a standalone "exit soaring" command.
14     _soaring_local_override = false;
15     _soaring_forbidden_latched = false;
16     PX4_INFO("Autosoaring: _glide ended -> AUTO_MISSION");
17 }
```

Why is this needed? In *thermal* mode the aircraft is in `AUTO_LOITER`; the thermal-exit block (Step 8) must explicitly switch back to `AUTO_MISSION`. In *glide* mode the aircraft stays in `AUTO_MISSION` throughout, but the navigator has been receiving continuous `DO_SET_MODE(AUTO_MISSION)` heartbeats from the 1s refresh. When glide ends the navigator is already in `AUTO_MISSION` — yet republishing the command ensures the navigator re-evaluates its setpoints and resumes waypoint tracking immediately, rather than coasting on the last glide setpoint.

Guard: `soaring_was_glide_only`. The flag `_soaring_was_glide` is updated at the end of each `control_auto()` cycle:

```
1  _soaring_was_glide = effective_glide;
2  _soaring_was_thermal = effective_thermal;
3  _soaring_was_bank_thermal = effective_thermal_bank;
```

The `!_soaring_was_thermal` guard prevents a double fire when a thermal-loiter ends and simultaneously `effective_glide` is false: the thermal-exit block (Step 8) already handles that case.

Interaction with `vehicle_command_poll()`. Clearing `_soaring_local_override` before publishing ensures that when the published `DO_SET_MODE` loops back through `vehicle_command_poll` it is processed as a normal mode switch and does not trigger the “standalone exit soaring” branch (which would call `set_gliding_mode_enabled(false)` redundantly).

30 Change 5: Mode 4 — Complete NPFG Bypass

30.1 Why Bypass

If the inner switch runs `control_auto_loiter()` first:

- NPFG computes `getAirspeedRef()` based on loiter geometry and wind.
- This modified airspeed propagates through `adapt_airspeed_setpoint()` and into TECS.
- TECS pitch then tracks the geometry-contaminated airspeed, not the polar or fixed glide airspeed.
- Result: airspeed oscillations and pitch-roll coupling.

30.2 Implementation

```
1  if (effective_thermal_bank) {
2      // Skip the switch entirely
3      tecs_update_pitch_throttle(
4          control_interval, current_altitude,
5          _param_fw_glide_airspeed.get(), // pure glide speed
6          radians(_param_fw_p_lim_min.get()),
7          radians(_param_fw_p_lim_max.get()),
8          _param_fw_thr_min.get(), _param_fw_thr_max.get(),
9          _param_sinkrate_target.get(), _param_climbrate_target.get(),
10         false);
11
12     const float bank_sign =
13         _autosoaring_control.loiter_clockwise ? 1.f : -1.f;
14     const float bank_rad =
15         bank_sign * radians(fabsf(_soaring_bank_angle_cmd_deg));
16     const Quatf q(Eulerf(bank_rad, get_tecs_pitch(), _yaw));
17     q.copyTo(_att_sp.q_d);
18     _att_sp.thrust_body[0] = get_tecs_thrust(); // = 0 (glide)
19
20 } else {
21     switch (position_sp_type) { /* standard dispatch */ }
22 }
```

The quaternion $\mathbf{q}^*$ encodes exactly three independently sourced values:

Component	Source
ϕ^*	$\pm \phi_{\text{cmd}} $ (companion bank angle)
θ^*	<code>get_tecs_pitch()</code> (glide speed hold)
ψ	current measured yaw (inner loop handles yaw coordination)

31 Change 6: Symmetric Throttle Ramps

31.1 Physical Motivation

Entry (powered $\rightarrow$ glide), instant cut:

- Propwash instantly collapses, reducing wing root dynamic pressure.
- Nose-down pitching moment, then TECS corrects: ± 1.5 m/s airspeed transient.

Exit (glide $\rightarrow$ powered), instant restore:

- Propwash surge + pitch integrator bias: 40–50 deg/s spike.

Both transients resolved by linear ramps of duration $T = \text{FW_GLIDE_RAMP_T}$ (default 3 s).

31.2 Pre-Glide Thrust Cache

Before glide starts, the current TECS thrust is saved:

```
1 if (!currently_soaring && !_landed)
2   _soaring_pre_glide_thrust = get_tecs_thrust();
```

31.3 Entry Ramp (glide starts)

On glide entry, `_soaring_entry_ramp_start_us` is set. Each cycle during entry:

$$\delta_t^{\text{entry}}(t) = \delta_{t,\text{pre}} \cdot \max\left(1 - \frac{t - t_s}{T}, 0\right) \quad (28)$$

After T seconds: $\delta_t^{\text{entry}} = 0$.

31.4 Exit Ramp (glide ends)

On glide exit, `_soaring_ramp_start_us` is set. Scale factor:

$$s_{\text{exit}}(t) = \min\left(\frac{t - t_e}{T}, 1\right) \quad (29)$$

31.5 Final Throttle Selection

Condition	Thrust formula	Ramp active?
Idle setpoint	0	No
Currently soaring, entry ramp active	$\delta_{t,\text{pre}}(1 - t/T)$	Entry
Currently soaring, ramp done	0	None (steady glide)
Not soaring, exit ramp active	$\delta_{t,\text{TECS}} \cdot s_{\text{exit}}$	Exit
Not soaring, normal powered	$\delta_{t,\text{TECS}}$	None
Landed	$\min(\delta_{t,\text{idle}}, 1)$	No

31.6 Physical Impact

Metric	Before ramp	After ramp
Entry airspeed deviation	~ 1.5 m/s	$\lesssim 0.4$ m/s
Exit pitch-rate spike	40–50 deg/s	$\lesssim 15$ deg/s
Throttle gradient	Step	$\leq 0.33 \text{ s}^{-1}$ (over $T = 3$ s)

32 Complete Modification Summary

#	Function / Feature	Original	Modified
1	Run() DDS poll	Not present	DDS poll + inhibit/watchdog
2	DDS authority	N/A	CLI overrides DDS; 5 s inhibit on <code>soar:off</code>
3	Staleness watchdog	N/A	2 s silence $\Rightarrow$ SOARING_OFF
4	vehicle_command_processed CUSTOM_0	Unhandled	Encodes modes 0–4
5	DO_SET_MODE handling	Ignored	Pairing detection + exit logic
6	parameters_update()	Push FW params	Also push glide/polar airspeed
7	control_auto()	Mode switch only	Full soaring dispatch at top
8	Altitude floor	None	Latch below FW_ALT_MIN
9	Altitude ceiling	None	Hysteresis at FW_ALT_MAX
10	Nav (glide)	Not commanded	DO_SET_MODE heartbeat 1 s
11	Nav (thermal)	Not commanded	DO_REPOSITION once (pinned)
12	Loiter direction	N/A	Signed radius, resend on flip
13	Pre-roll safety	None	5% above banked stall
14	Mission index	Not saved	Latched; restored on exit
15	TECS glide flag	Never set	Set for modes 1–4
16	Airspeed setpoint	Mission speed	Mode-dependent (fixed/polar/hold)
17	Mode 4 roll	None	Direct $\pm \phi_{\text{cmd}} $, NPFG bypassed
18	Throttle entry	Instant cut	Linear ramp T_{ramp}
19	Throttle exit	Instant restore	Linear ramp T_{ramp}